Project Documentation: GOG Games — Rating Prediction & EDA

Dataset: https://www.kaggle.com/datasets/lunthu/gog-com-video-games-dataset/data

About the data:

No. of columns in Raw Data: 51

No. of rows in Raw Data: 10896

Why is Review prediction important?

Game review prediction is critical because it directly influences a game's market success and shapes ongoing development. By forecasting user ratings before or immediately upon launch, developers can optimize marketing strategies, address critical player feedback, and refine gameplay to ensure long-term engagement and profitability.

In this dataset, the target column is “overallAvgRating” (a continuous score from 0–5). For this project, the rating was bucketed into three quartile classes (rating_class ;Poor,Average And Excellent) and predicted as a multi-class classification problem.

1. Import the necessary libraries:

   ```
   import pandas as pd
   import seaborn as sns
   import matplotlib.pyplot as plt
   pd.set_option('display.max_columns', 500)
   ```

2. Import Data: df = pd.read_csv(“filepath”)
   Import Data: df_orignal = pd.read_csv(“filepath”) # To use later in EDA

3. Get a feel for the data using df.head() and df.tail()

4. Find shape and size of the data(use df.shape or df.size)

5. Use Info to see the name of each column the total non-null values

```
<class 'pandas.DataFrame'>
RangeIndex: 10896 entries, 0 to 10895
Data columns (total 51 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   id                         10896 non-null  int64
 1   developer                  10896 non-null  str
 2   publisher                  10896 non-null  str
 3   gallery                    10896 non-null  str
 4   video                      6961 non-null   str
 5   supportedOperatingSystems  10896 non-null  str
 6   genres                     10896 non-null  str
 7   globalReleaseDate          10393 non-null  float64
 8   isTBA                      10896 non-null  bool
 9   isDiscounted_main          10896 non-null  bool
 10  isInDevelopment            10896 non-null  bool
 11  releaseDate                10393 non-null  float64
 12  availability               10896 non-null  str
 13  salesVisibility            10896 non-null  str
 14  buyable                    10896 non-null  bool
 15  title                      10896 non-null  str
 16  image                      10896 non-null  str
 17  url                        10896 non-null  str
 18  supportUrl                 10896 non-null  str
 19  forumUrl                   10896 non-null  str
 20  worksOn                    10896 non-null  str
 21  category                   10896 non-null  str
 22  originalCategory           10896 non-null  str
 23  rating                     10896 non-null  int64
 24  type                       10896 non-null  int64
 25  isComingSoon               10896 non-null  bool
 26  isPriceVisible             10896 non-null  bool
 27  isMovie                    10896 non-null  bool
 28  isGame                     10896 non-null  bool
 29  slug                       10896 non-null  str
 30  isWishlistable             10896 non-null  bool
 31  ageLimit                   10896 non-null  int64
 32  boxImage                   10896 non-null  str
 33  isMod                      10896 non-null  bool
 34  currency                   10896 non-null  str
 35  amount                     10896 non-null  float64
 36  baseAmount                 10896 non-null  float64
 37  finalAmount                10896 non-null  float64
 38  isDiscounted               10896 non-null  bool
 39  discountPercentage         10896 non-null  int64
 40  discountDifference         10896 non-null  float64
 41  isFree                     10896 non-null  bool
 42  discount                   10896 non-null  int64
 43  promoId                    7908 non-null   str
 44  filteredAvgRating          10896 non-null  float64
 45  overallAvgRating           10896 non-null  float64
 46  reviewCount                10896 non-null  int64
 47  isReviewable               10896 non-null  bool
 48  reviewPages                10896 non-null  int64
 49  dateGlobal                 10393 non-null  str
 50  dateReleaseDate            10393 non-null  str
dtypes: bool(13), float64(8), int64(8), str(22)
memory usage: 21.1 MB
```

Observations:

- genres, worksOn, availability, salesVisibility are stored as strings that look like lists/dicts — ISSUE
- developer and publisher are raw text with hundreds of unique values — ISSUE
- globalReleaseDate is a Unix timestamp, not a usable year — ISSUE
- Several columns are pure metadata (image/gallery/video URLs, forum/support links, slugs) — not useful for prediction and were dropped
- Several NULL values — ISSUE

Checking for null and duplicated Values:

Checking for null values using **df.isnull().sum()**:

```
id                          0
developer                   0
publisher                   0
gallery                     0
video                    3935
supportedOperatingSystems   0
genres                      0
globalReleaseDate         503
isTBA                       0
isDiscounted_main           0
isInDevelopment             0
releaseDate               503
availability                0
salesVisibility             0
buyable                     0
title                       0
image                       0
url                         0
supportUrl                  0
forumUrl                    0
worksOn                     0
category                    0
originalCategory            0
rating                      0
type                        0
isComingSoon                0
isPriceVisible              0
isMovie                     0
isGame                      0
slug                        0
isWishlistable              0
ageLimit                    0
boxImage                    0
isMod                       0
currency                    0
amount                      0
baseAmount                  0
finalAmount                 0
isDiscounted                0
discountPercentage          0
discountDifference          0
isFree                      0
discount                    0
promoId                  2988
filteredAvgRating           0
overallAvgRating            0
reviewCount                 0
isReviewable                0
reviewPages                 0
dateGlobal                503
dateReleaseDate           503
dtype: int64
```

Checking for duplicate rows using df[df.duplicated(keep=False)]:

O/P: no fully duplicated rows were found.

Handling nulls:

- Remaining nulls (mostly in the timestamp columns and video, which were being dropped anyway): handled by a single df.dropna() after the irrelevant columns were removed.

Dropping columns that are irrelevant:

- Columns that contain metadata, duplicated columns were removed since they don't have any significance.

- RUN:

```
df.drop(columns=[
    'id','video','gallery','image','dateReleaseDate','releaseDate',
    'dateGlobal','boxImage','url','title','slug','supportUrl','forumUrl',
    'supportedOperatingSystems','originalCategory','baseAmount','amount',
    'currency','rating','filteredAvgRating','discountPercentage',
    'isDiscounted','isDiscounted_main',’promoID’
], inplace=True)
```

6. Data cleaning
   a. **Genres — multi-label binarization**
      - genres is a list of tags per game (e.g. ['Role-playing', 'Action', 'Sci-fi']), so a single one-hot column per category would be wrong — a game can belong to several genres at once. MultiLabelBinarizer was used to create one binary column per genre (54 genre columns total).

      - RUN:

```
from sklearn.preprocessing import MultiLabelBinarizer
import ast
mlb = MultiLabelBinarizer()
df['genres'] = df['genres'].apply(ast.literal_eval)
df_genres = pd.DataFrame(
   mlb.fit_transform(df['genres']),
   columns = mlb.classes_,
   index = df.index)
```

```
df = pd.concat([df.drop(columns=['genres']),df_genres],axis=1)
```

b. Platform support (worksOn)

- **worksOn is a dict of {Windows, Mac, Linux: bool}. This was unpacked into three separate binary columns.**
- RUN:

```
df['worksOn'] = df['worksOn'].apply(ast.literal_eval)
df['Windows'] = df['worksOn'].str['Windows']
df['Mac'] = df['worksOn'].str['Mac']
df['Linux'] = df['worksOn'].str['Linux']
df.drop('worksOn',axis=1,inplace=True)
```

c. Availability

- availability is a dict with isAvailable / isAvailableInAccount flags — unpacked the same way, then all boolean columns (isAvailable, isAvailableInAccount, Windows, Mac, Linux, isTBA, isInDevelopment, buyable, isComingSoon, isPriceVisible, isMovie, isGame, isWishlistable, isMod, isFree, isReviewable) were cast to int so the model can use them.

d. **Frequency encoding for high-cardinality categoricals**

- Developer and publisherhave hundreds of unique text values, too many for one-hot encoding. Each was replaced with its own frequency count — i.e. how many games share that same developer/publisher/promo — which captures "is this a prolific studio" without exploding the column count.

- RUN:

```
df['developer'] = df['developer'].map(df['developer'].value_counts())
df['publisher'] = df['publisher'].map(df['publisher'].value_counts())
```

e. **Sale status**

- salesVisibility is a dict containing an isActive flag — extracted into a single salesActive binary column.
- RUN:

```
df['salesVisibility'] = df['salesVisibility'].apply(ast.literal_eval)
active = []
for i in df['salesVisibility']:
 active.append(i['isActive'])
df['salesActive'] = active
df.drop('salesVisibility',inplace=True,axis=1)
df['salesActive'] = df['salesActive'].astype(int)
```

f. **Category**
   - Unlike genres, each game has exactly one category (Adventure, Action, Strategy, etc.), so standard one-hot encoding (with drop_first=True to avoid redundancy) was appropriate here.

Observations:

- Final Data size: (9470, 90)

- Now we are ready to train our ML model; No non-binary encoded columns**,** no null values and no duplicated rows.

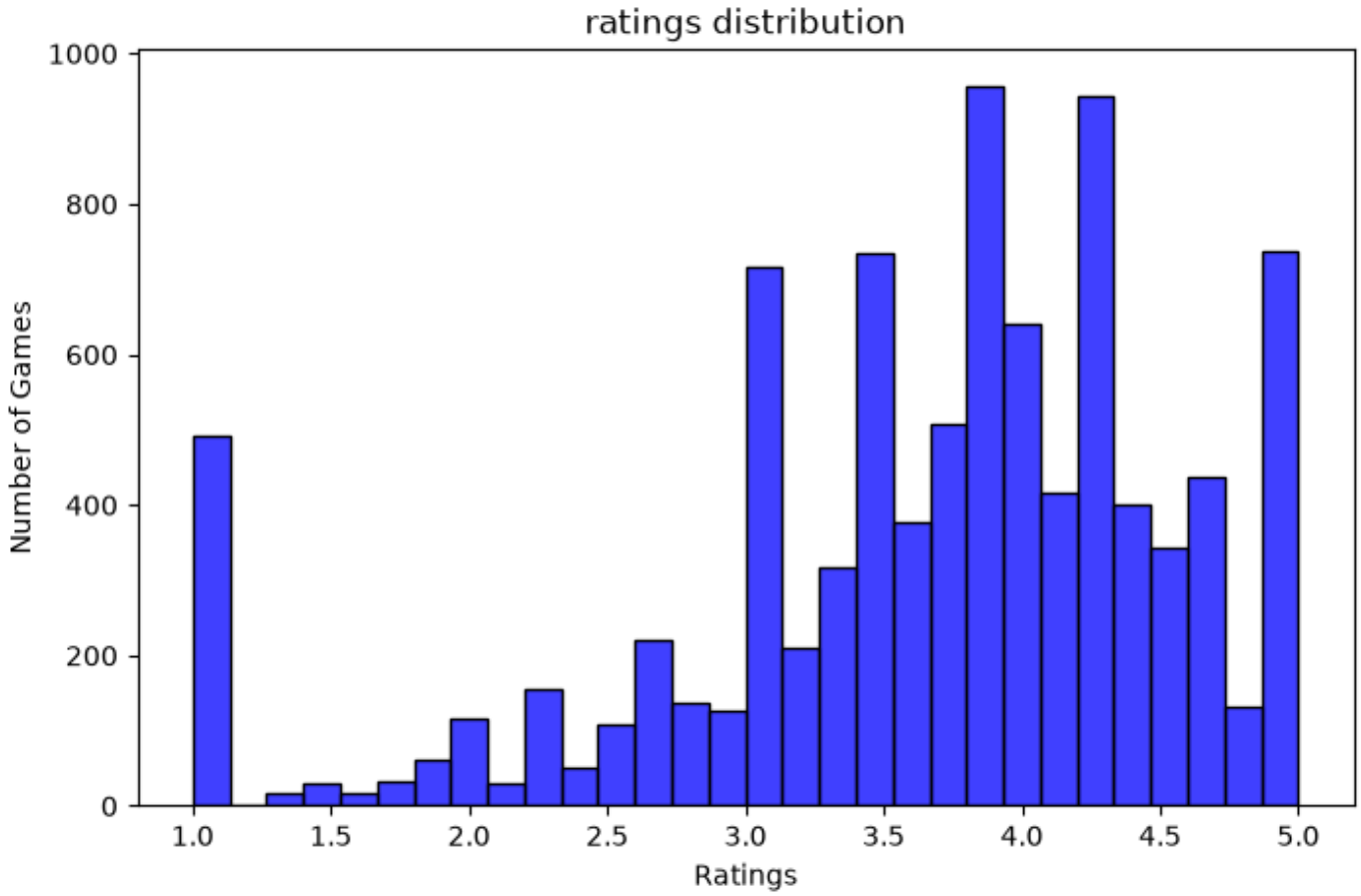
7. EDA(Exploratory Data Anaylyis):

Here, We need to see how some specific columns are related to game ratings using 4-5 graphs. We will do a comparison of the following:

a. Distribution of overall average ratings

RUN:

```
plt.figure(figsize=(8,5))
sns.histplot(df['overallAvgRating'],bins=30,color='Blue')
plt.xlabel("Ratings")
plt.ylabel("Number of Games")
plt.title("ratings distribution")
plt.show()
```



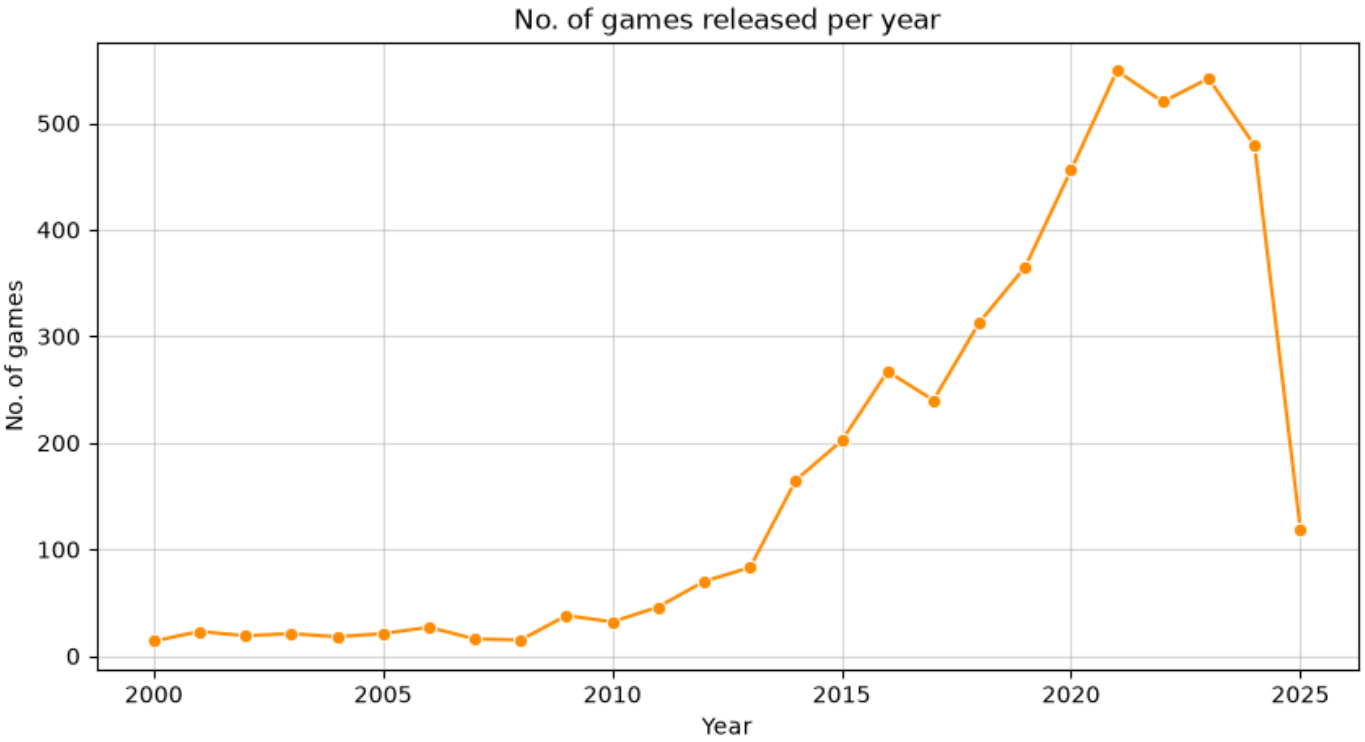
- Most games receive favorable ratings, indicating that the GOG catalog generally consists of well-reviewed titles.
- Very low-rated games are relatively uncommon.
- Users on the platform appear to rate games positively more often than negatively.

b. Games released per year

RUN:

```
release_years = pd.to_datetime(df_og['globalReleaseDate'],unit='s').dt.year
games_per_year = release_years.value_counts().sort_index()
games_per_year = games_per_year[games_per_year.index >= 2000]
plt.figure(figsize=(10,5))
sns.lineplot(x=games_per_year.index,y=games_per_year.values,marker='o',color
='darkorange')
plt.xlabel("Year")
plt.ylabel("No. of games")
plt.title("No. of games released per year")
plt.grid(alpha=0.5)
```

plt.show()

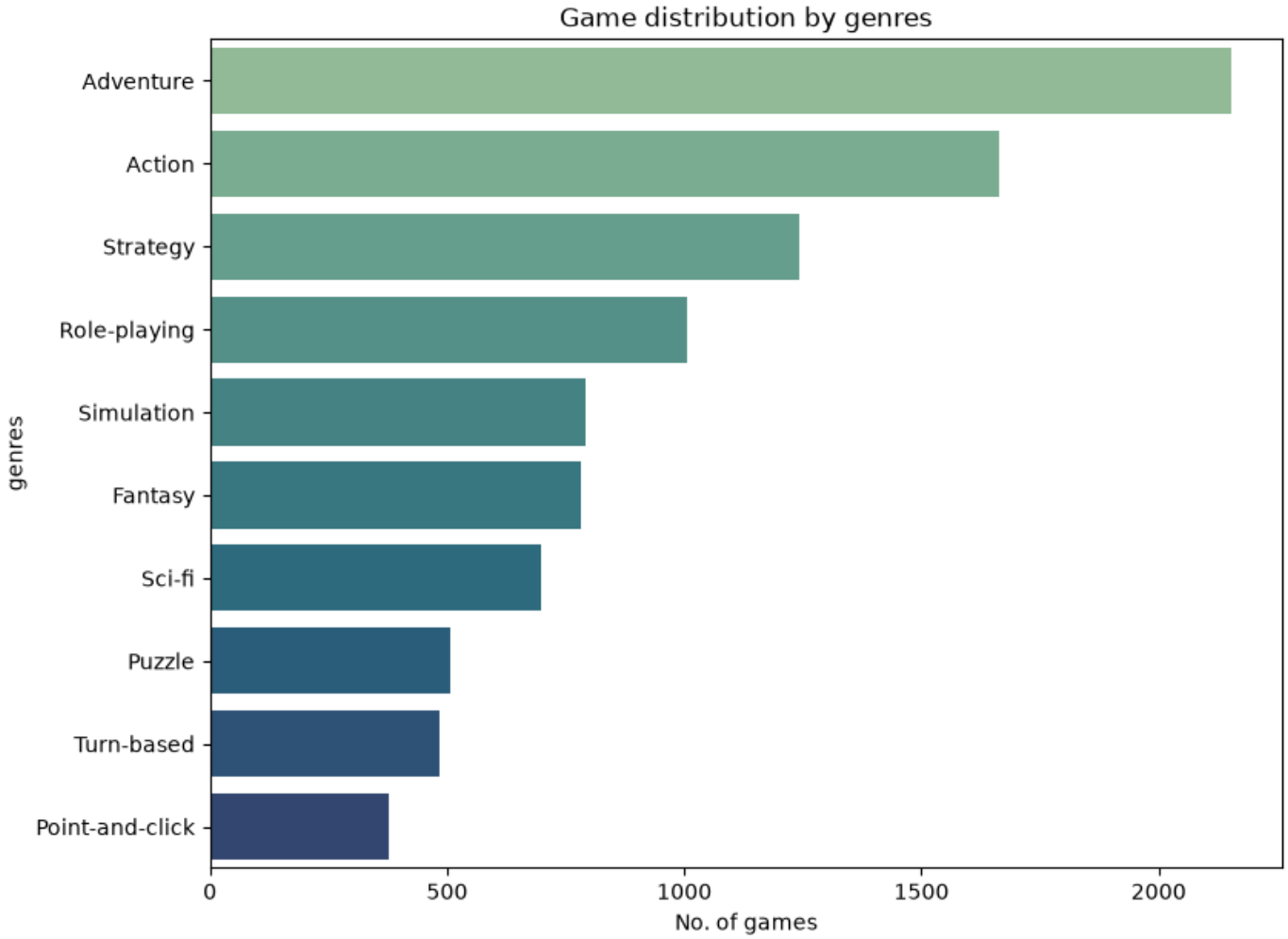


- The number of games released has generally increased over the years.
- Recent years contain a much larger selection of games than earlier years.
- The platform has expanded its game library significantly over time.

c. Top 10 Game Genres

RUN:

```
genre_series = df_og['genres'].apply(ast.literal_eval).explode()
top_genres = genre_series.value_counts().head(10)
plt.figure(figsize=(9,7))
sns.barplot(x=top_genres.values,y=top_genres.index,palette='crest')
plt.xlabel("Genres")
plt.xlabel("No. of games")
plt.title("Game distribution by genres")
plt.show()
```

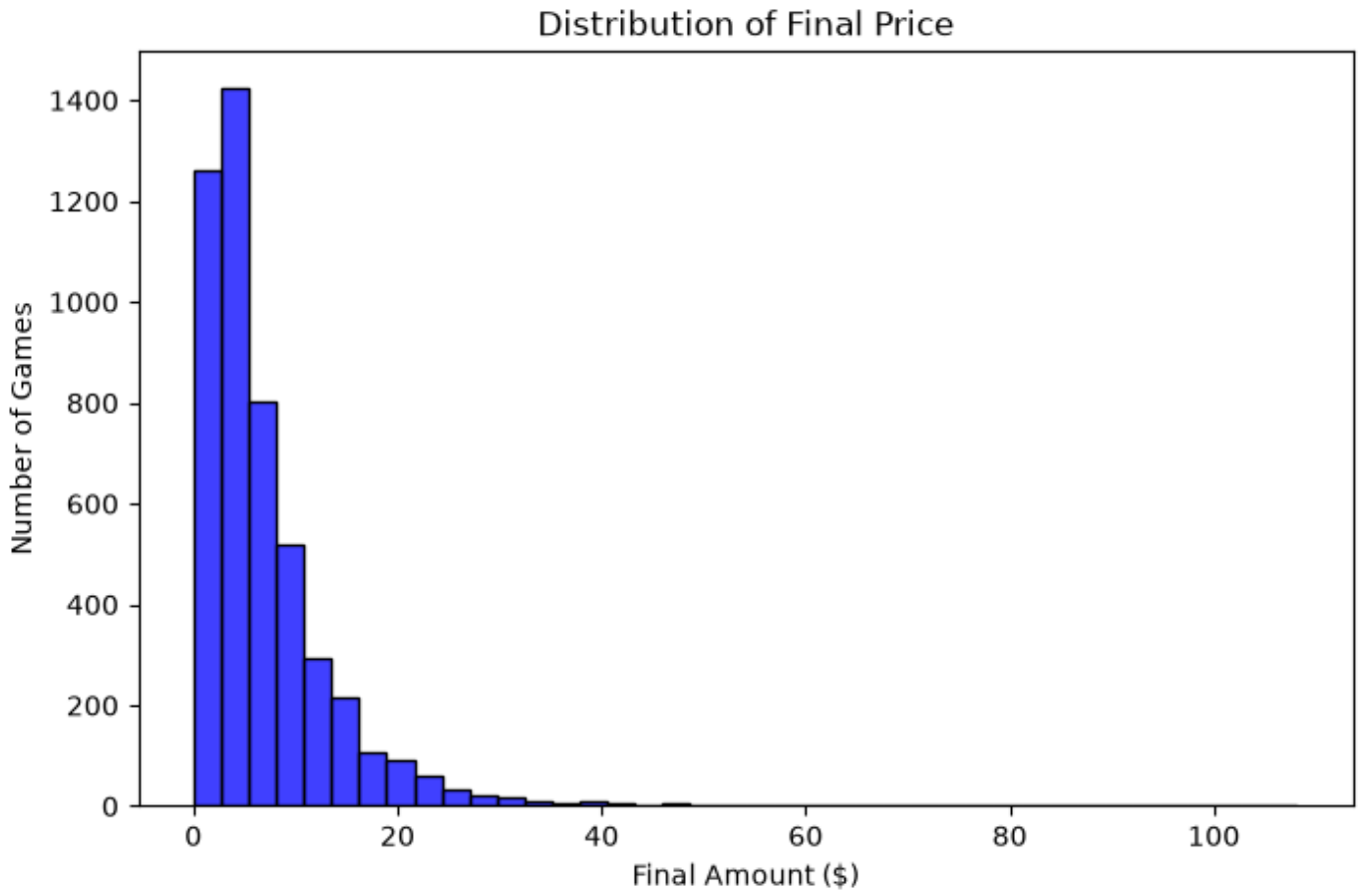


- A few genres dominate the platform's catalog.
- Action and Adventure-related games appear more frequently than niche genres.
- Players have a wide variety of games available, but certain genres are clearly more popular among developers.

d. Distribution of Final Price

RUN:

```
plt.figure(figsize=(8,5))
sns.histplot(df_og['finalAmount'], bins=40,color ='Blue')
plt.title('Distribution of Final Price')
plt.xlabel('Final Amount ($)')
plt.ylabel('Number of Games')
```

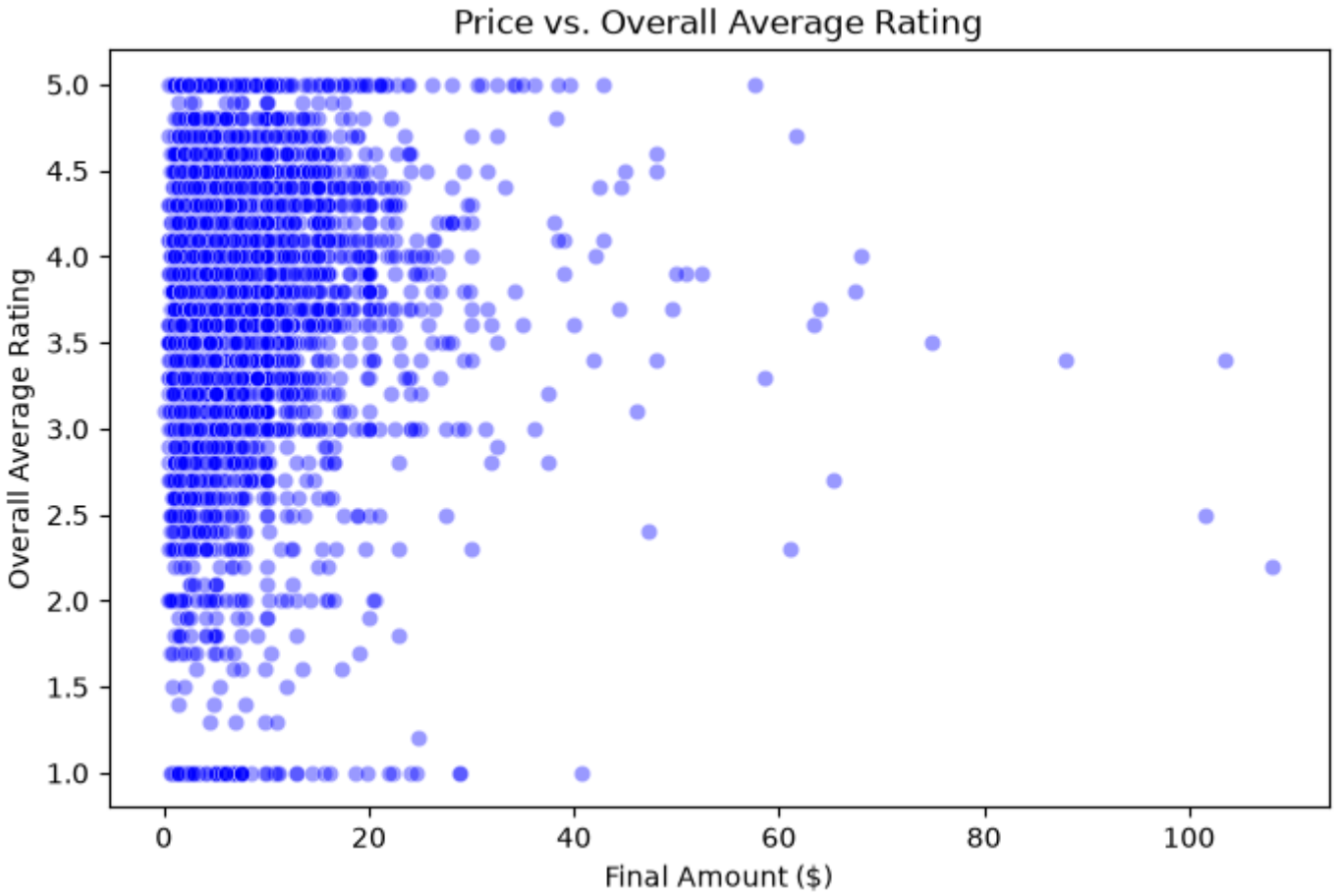


- Most games are priced in the lower price range.
- High-priced games are relatively uncommon.
- The platform offers a large number of affordable games, making it attractive to budget-conscious players.

e. Final Price vs Overall Average Rating

RUN:

```
filtered = df_og[df_og['overallAvgRating'] > 0]
plt.figure(figsize=(8,5))
sns.scatterplot(x=filtered['finalAmount'],
y=filtered['overallAvgRating'],alpha=0.4,color='Blue')
plt.title('Price vs. Overall Average Rating')
plt.xlabel('Final Amount ($)')
plt.ylabel('Overall Average Rating')
```

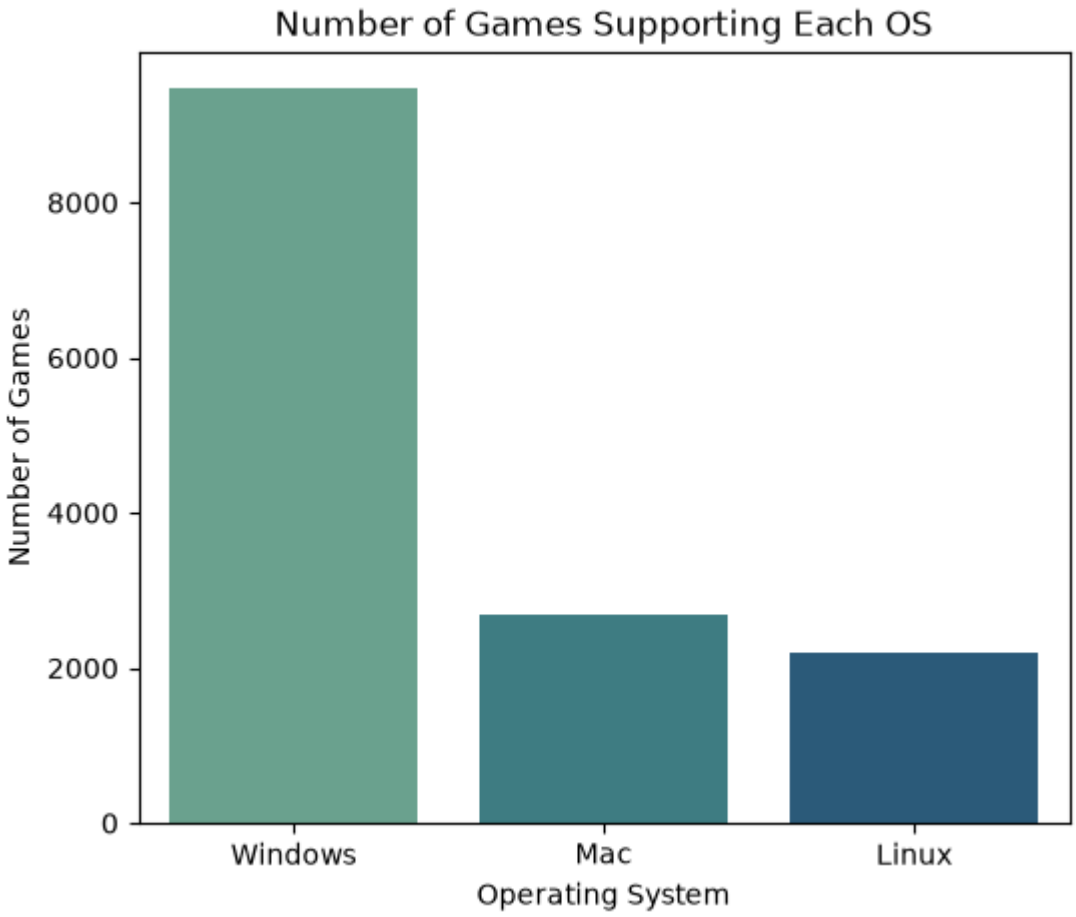


- Highly rated games can be found across different price ranges.
- Expensive games are not necessarily rated higher than cheaper games.
- Price alone does not appear to determine how positively a game is received.

f. Number of Games Supporting Each Operating System

RUN:

```
os_support = df[['Windows','Mac','Linux']].sum()
plt.figure(figsize=(6,5))
sns.barplot(x=os_support.index, y=os_support.values, palette='crest')
plt.title('Number of Games Supporting Each OS')
plt.xlabel('Operating System')
plt.ylabel('Number of Games')
```

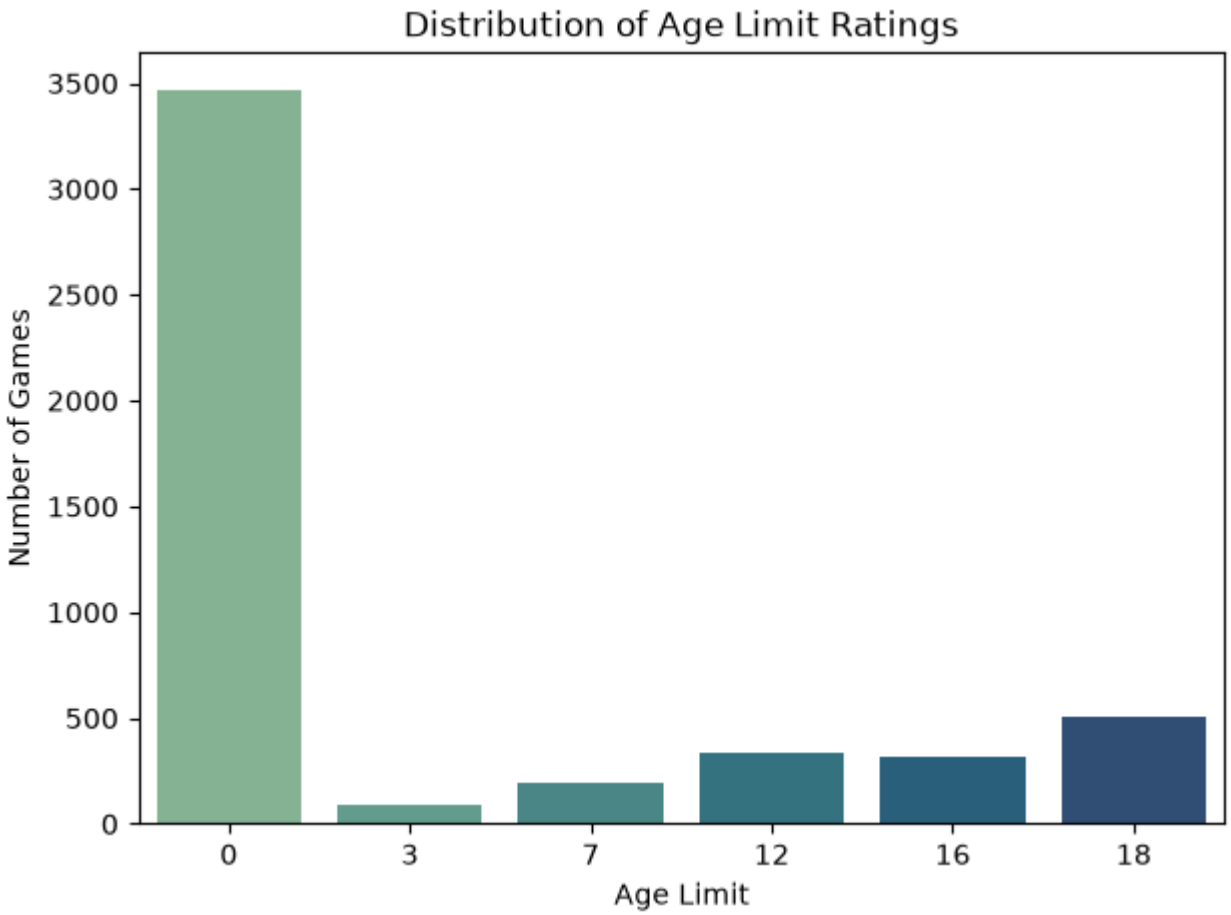


- Windows has the largest game support among all operating systems.
- macOS and Linux support is available for many games but is less common.
- Developers primarily target Windows while still offering compatibility for other platforms in selected titles.

g. Distribution of Age Limit Ratings

RUN:

```
plt.figure(figsize=(7,5))
sns.countplot(x=df_og['ageLimit'].sort_values(), palette='crest')
plt.title('Distribution of Age Limit Ratings')
plt.xlabel('Age Limit')
plt.ylabel('Number of Games')
plt.show()
```

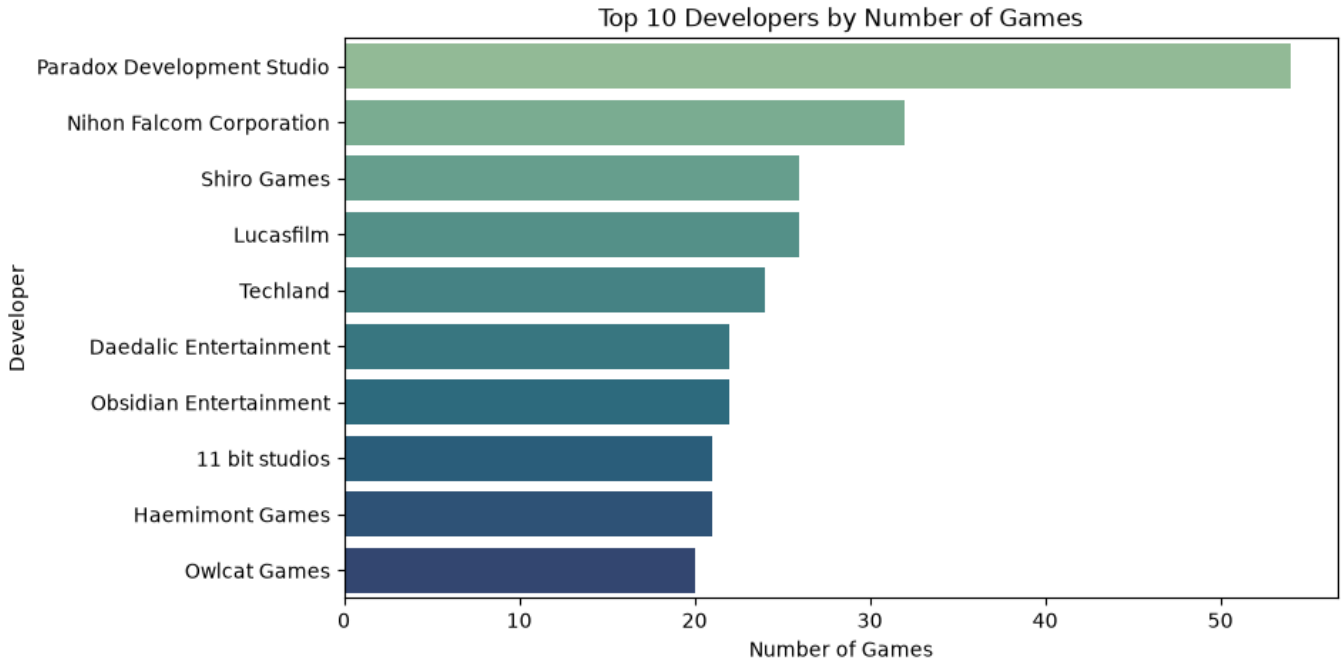


- Most games fall into a small number of age categories.
- Games suitable for teenagers and adults make up a significant portion of the catalogue.
- Very restrictive age ratings appear less frequently.

h. Top 10 Developers by Number of Games

RUN:

```
top_devs = df_og['developer'].value_counts().head(10)
plt.figure(figsize=(9,5))
sns.barplot(x=top_devs.values, y=top_devs.index, palette='crest')
plt.title('Top 10 Developers by Number of Games')
plt.xlabel('Number of Games')
plt.ylabel('Developer')
```



- A small group of developers contributes a large number of games to the platform.
- Most developers have only a few titles compared to the leading publishers.
- The catalog contains both established developers and many smaller studios.

8. Now we get into applying Random Forest algorithm onto this data:

   We first split the data(80%trainin,20%testing)

   ```
   from sklearn.ensemble import RandomForestClassifier
   from sklearn.model_selection import train_test_split

   x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
   ```

9. Now we are going to apply the Random forest algorithm for this data. x_train will be X variable and y_train will be the y variable.

RUN:

```
from sklearn.ensemble import RandomForestClassifier
RF_model=RandomForestClassifier(n_estimators=500,stratify=y,
random_state=42)
RF.fit(x_train,y_train)
RF_pred = RF.predict(x_test)
print(classification_report(y_test,RF_pred))
print(confusion_matrix(y_test,RF_pred))
```

10.Now we need to find predictions based on test values of X:

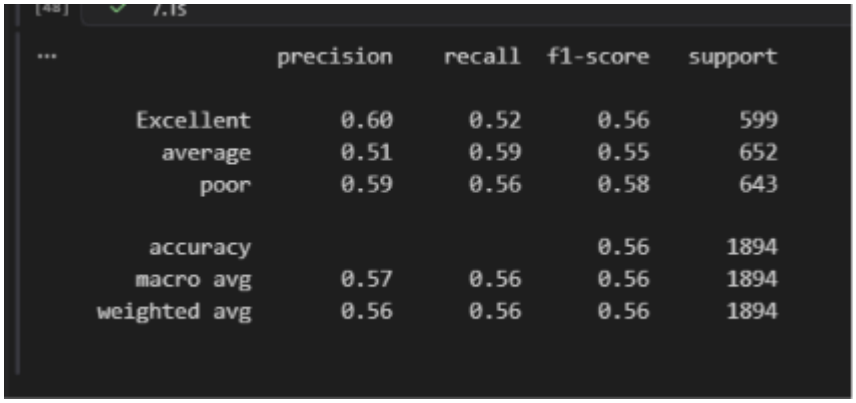
RUN:

```
Y_pred = RF_model.predict(x_test)
```

11. Then we need to check the accuracy of our model using metrics:

RUN:

```
from sklearn.metrics import classification_report
print(classification_report(y_test,pred))
```

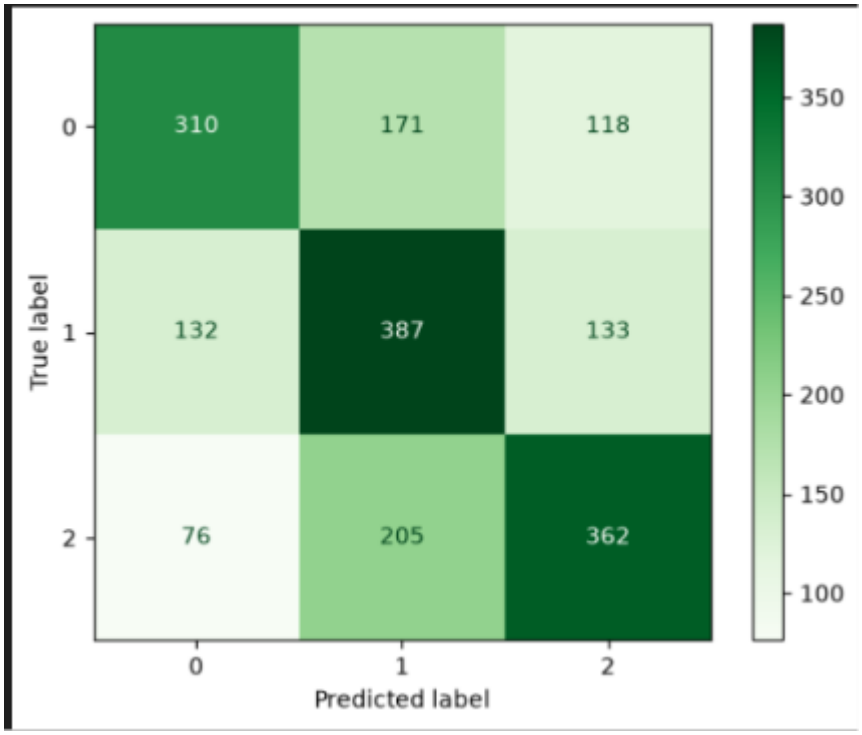


	precision	recall	f1-score	support
Excellent	0.60	0.52	0.56	599
average	0.51	0.59	0.55	652
poor	0.59	0.56	0.58	643
accuracy			0.56	1894
macro avg	0.57	0.56	0.56	1894
weighted avg	0.56	0.56	0.56	1894

12. Now we display the confusion matrix:

RUN:

```
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay
cm = confusion_matrix(y_test,RF_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap=plt.cm.Greens)
plt.show()
```



Observations:

a. **Poor:**
   - **310 were correctly predicted as poor**
   - **171 were incorrectly predicted as Average**
   - **118 were incorrectly predicted as Excellent**

b. **Average:**
   - **132 were incorrectly predicted as poor**
   - **387 were correctly predicted as Average**
   - **133 were incorrectly predicted as Excellent**

c. **Excellent:**
   - **76 were incorrectly predicted as poor**
   - **205 were incorrectly predicted as Average**
   - **362 were correctly predicted as Excellent**